

University of Science and Technology Chittagong (USTC)

Department of Computer Science and Engineering



Lab Report-2

Course Title: Computer Graphics and Image Processing Lab

Course Code: CSE 418

Submitted To

Umme Aiman Jannat

Lecturer

CSE, FSET, USTC

Submitted By

Name: **Zahid Al Noor Himanta**

Roll No: **23070152**

Registration No: **1058**

Batch: **41 (Section- B, CSE)**

7th Semester

Experiment 1: Implementation of Direct Line Algorithm

Source Code:

```
import matplotlib.pyplot as plt

def direct_line(x1, y1, x2, y2):
    pixels = []

    print("\n--- Direct line equation method ---")
    print("Starting Point:", (x1, y1))
    print("Ending Point:", (x2, y2))

    dx = x2 - x1
    dy = y2 - y1

    # For only vertical Line
    if dx == 0:
        print("Case: Vertical Line.\n")
        print("Slope is undefined.\n")

        start_y = min(y1, y2)
        end_y = max(y1, y2)

        for y in range(start_y, end_y + 1):
            pixels.append((x1, y))

    # For horizontal or non-vertical Line
    else:
        m = dy / dx
        b = y1 - m * x1

        print("Slope:", m)
        print("Y-intercept:", b)
        # Shallow Line
        if abs(m) <= 1:
            print("Case: Shallow Line.\n")

            start_x = min(x1, x2)
            end_x = max(x1, x2)

            for x in range(start_x, end_x + 1):
                y = m * x + b
                round_y = round(y)

                pixels.append((x, round_y))

                print("x =", x)
                print("y =", round_y)
                print("pixel ->", (x, round_y))

            # Step / Steep Line
            else:
                print("Case: Step Line.\n")

                start_y = min(y1, y2)
                end_y = max(y1, y2)

                for y in range(start_y, end_y + 1):
                    x = (y - b) / m
                    round_x = round(x)

                    pixels.append((round_x, y))

    print("x =", round_x)
    print("y =", y)
    print("pixel ->", (round_x, y))

    return pixels

### Main Program ###

# Enter Coordinates

x1 = int(input("Enter x1: "))
y1 = int(input("Enter y1: "))
x2 = int(input("Enter x2: "))
y2 = int(input("Enter y2: "))

## calling function
pixels=direct_line(x1,y1,x2,y2)

import matplotlib.pyplot as plt

# Plotting Pixels

x_pixels = [p[0] for p in pixels]
y_pixels = [p[1] for p in pixels]

plt.figure(figsize=(8, 6))

# Ideal Mathematical Line
plt.plot(
    [x1, x2],
    [y1, y2],
    label="Ideal Mathematical Line"
)

# Generated Pixels
plt.scatter(
    x_pixels,
    y_pixels,
    s=100,
    marker="s",
    color="red",
    label="Generated Pixels"
)

# End Points
plt.scatter(
    [x1, x2],
    [y1, y2],
    s=150,
    marker="o",
    color="blue",
    label="End Points"
)
```

```
# Pixel coordinate labels
for x, y in pixels:
    plt.text(
        x + 0.08,
        y + 0.08,
        f"({x},{y})",
        fontsize=10
    )

plt.xticks(range(min(x1, x2) - 1, max(x1, x2) + 2))
plt.yticks(range(min(y1, y2) - 1, max(y1, y2) + 2))

plt.grid(True)
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.title("Direct Line Algorithm")
plt.legend()
plt.show()
```

Output:

```
Enter x1: 0
Enter y1: 0
Enter x2: 8
Enter y2: 19
```

--- Direct line equation method ---

Starting Point: (0, 0)

Ending Point: (8, 19)

Slope: 2.375

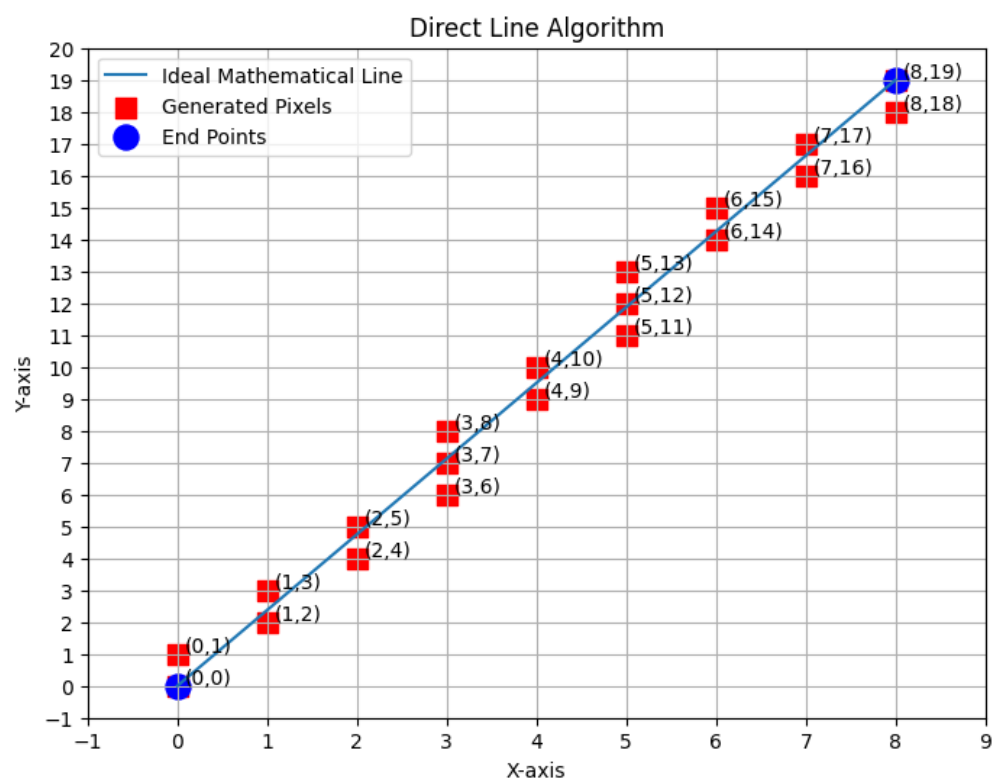
Y-intercept: 0.0

Case: Step Line.

```
x = 0
y = 0
pixel -> (0, 0)
x = 0
y = 1
pixel -> (0, 1)
x = 1
y = 2
pixel -> (1, 2)
x = 1
y = 3
pixel -> (1, 3)
x = 2
y = 4
pixel -> (2, 4)
x = 2
y = 5
pixel -> (2, 5)
x = 3
y = 6
pixel -> (3, 6)
x = 3
y = 7
pixel -> (3, 7)
x = 3
y = 8
pixel -> (3, 8)
```

```
x = 4
y = 9
pixel -> (4, 9)
x = 4
y = 10
pixel -> (4, 10)
x = 5
y = 11
pixel -> (5, 11)
x = 5
y = 12
pixel -> (5, 12)
x = 5
y = 13
pixel -> (5, 13)
x = 6
y = 14
pixel -> (6, 14)
```

```
x = 6
y = 15
pixel -> (6, 15)
x = 7
y = 16
pixel -> (7, 16)
x = 7
y = 17
pixel -> (7, 17)
x = 8
y = 18
pixel -> (8, 18)
x = 8
y = 19
pixel -> (8, 19)
```



Experiment 2: Implementation of DDA Scan Conversion

Source Code:

```
import matplotlib.pyplot as plt

def DDA_line(x1, y1, x2, y2):
    pixels = []

    print("\n--- DDA Line Drawing Algorithm ---")
    print("Starting Point:", (x1, y1))
    print("Ending Point:", (x2, y2))

    # Calculate delta x and delta y
    dx = x2 - x1
    dy = y2 - y1

    print("Delta X =", dx)
    print("Delta Y =", dy)

    # Vertical Line
    if dx == 0:
        print("\nCase: Vertical Line")

        step_y = 1 if dy > 0 else -1

        x = x1
        y = y1

        pixels.append((round(x), round(y)))

        while y != y2:
            y = y + step_y
            pixels.append((round(x), round(y)))

            print("x =", round(x))
            print("y =", round(y))
            print("pixel ->", (round(x), round(y)))
        else:
            # Calculate slope
            m = dy / dx

            print("Slope (m) =", m)

            # Case 1: |delta x| >= |delta y|
            if abs(dx) >= abs(dy):
                print("\nCase 1: |Delta X| >= |Delta Y|")

                # Delta X = 1 or -1
                step_x = 1 if dx > 0 else -1

                # x(i+1) = xi + Delta X
                # y(i+1) = yi + m * Delta X

                x = x1
                y = y1

                pixels.append((round(x), round(y)))

                while round(x) != x2:
                    x = x + step_x
                    y = y + m * step_x

                    pixels.append((round(x), round(y)))

                    print("x =", round(x))
                    print("y =", round(y))
                    print("pixel ->", (round(x), round(y)))
            else:
                # Case 2: |delta x| < |delta y|
                print("\nCase 2: |Delta X| < |Delta Y|")

                # Delta Y = 1 or -1
                step_y = 1 if dy > 0 else -1

                # x(i+1) = xi + Delta Y / m
                # y(i+1) = yi + Delta Y

                x = x1
                y = y1

                pixels.append((round(x), round(y)))

                while round(y) != y2:
                    y = y + step_y
                    x = x + step_y / m

                    pixels.append((round(x), round(y)))

                    print("x =", round(x))
                    print("y =", round(y))
                    print("pixel ->", (round(x), round(y)))

    return pixels

### Main Program ###

x1 = int(input("Enter x1: "))
y1 = int(input("Enter y1: "))
x2 = int(input("Enter x2: "))
y2 = int(input("Enter y2: "))

# Call DDA function
pixels = DDA_line(x1, y1, x2, y2)

### Plotting ###

x_pixels = [p[0] for p in pixels]
y_pixels = [p[1] for p in pixels]

plt.figure(figsize=(8, 6))

# Ideal mathematical line
plt.plot(
    [x1, x2],
    [y1, y2],
    label="Ideal Mathematical Line"
)

# Generated pixels
plt.scatter(
    x_pixels,
    y_pixels,
    s=100,
    marker="s",
    color="red",
    label="DDA Generated Pixels"
)
```

```

# End points
plt.scatter(
    [x1, x2],
    [y1, y2],
    s=150,
    marker="o",
    color="blue",
    label="End Points"
)

# Pixel coordinate labels
for x, y in pixels:
    plt.text(
        x + 0.08,
        y + 0.08,
        f"({x},{y})",
        fontsize=10
    )

plt.xticks(range(min(x1, x2) - 1, max(x1, x2) + 2))
plt.yticks(range(min(y1, y2) - 1, max(y1, y2) + 2))

plt.grid(True)
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.title("DDA Scan Conversion")
plt.legend()
plt.show()

```

Output:

Enter x1: 2
 Enter y1: 5
 Enter x2: 14
 Enter y2: 15

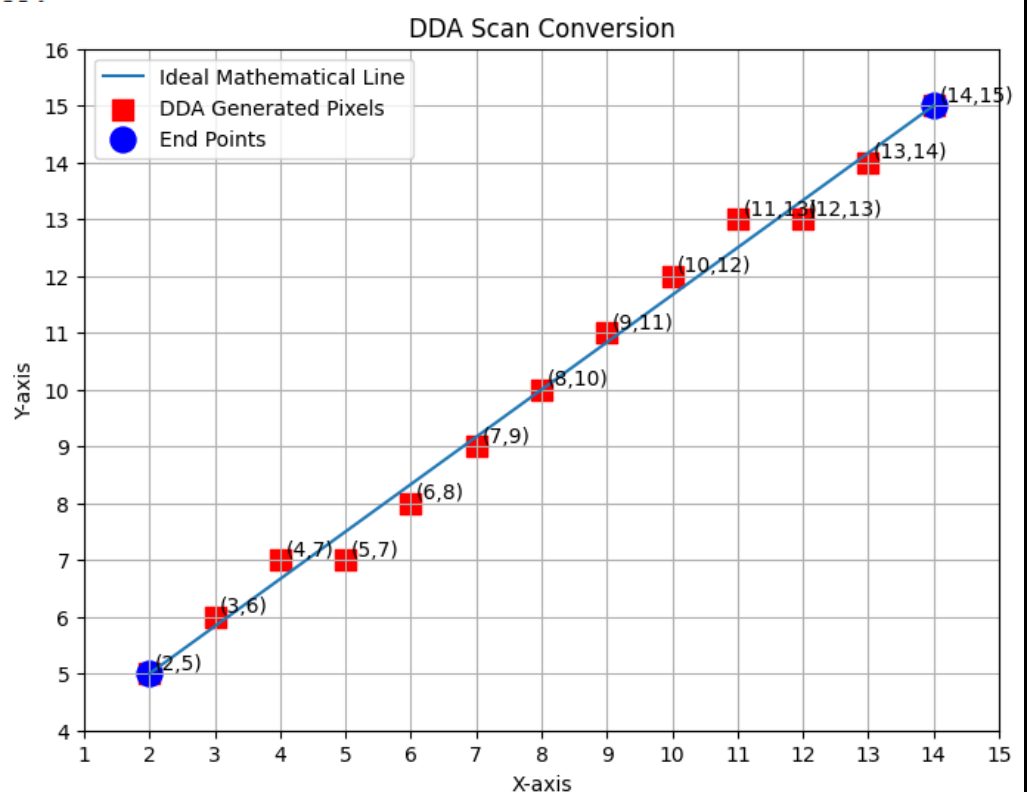
x = 10
 y = 12
 pixel -> (10, 12)
 x = 11
 y = 13
 pixel -> (11, 13)
 x = 12
 y = 13
 pixel -> (12, 13)

x = 13
 y = 14
 pixel -> (13, 14)
 x = 14
 y = 15
 pixel -> (14, 15)

--- DDA Line Drawing Algorithm ---
 Starting Point: (2, 5)
 Ending Point: (14, 15)
 Delta X = 12
 Delta Y = 10
 Slope (m) = 0.8333333333333333 ---

Case 1: $|\Delta x| \geq |\Delta y|$

x = 3
 y = 6
 pixel -> (3, 6)
 x = 4
 y = 7
 pixel -> (4, 7)
 x = 5
 y = 7
 pixel -> (5, 7)
 x = 6
 y = 8
 pixel -> (6, 8)
 x = 7
 y = 9
 pixel -> (7, 9)
 x = 8
 y = 10
 pixel -> (8, 10)
 x = 9
 y = 11
 pixel -> (9, 11)



Experiment 3: Implementation of Bresenham's Straight Line Algorithm.

Source Code:

```
import matplotlib.pyplot as plt

def bresenham_line(x1, y1, x2, y2):
    pixels = []

    print("\n--- Bresenham's Straight Line Algorithm ---")
    print("Starting Point:", (x1, y1))
    print("Ending Point:", (x2, y2))

    # Calculate Delta X and Delta Y
    dx = x2 - x1
    dy = y2 - y1

    print("Delta X =", dx)
    print("Delta Y =", dy)

    # Initial Decision Parameter
    p = 2 * dy - dx
    print("Initial Decision Parameter P =", p)

    # Starting point
    x = x1
    y = y1
    pixels.append((x, y))
    print("\nStarting Pixel ->", (x, y))

    # Repeat until ending point
    while x < x2:

        # Case 1: P < 0
        if p < 0:
            print("\nCase 1: P < 0")
            x = x + 1
            y = y
            #  $P(k+1) = P(k) + 2 \Delta Y$ 
            p = p + 2 * dy

        # Case 2: P >= 0
        else:
            print("\nCase 2: P >= 0")
            x = x + 1
            y = y + 1
            #  $P(k+1) = P(k) + 2 \Delta Y - 2 \Delta X$ 
            p = p + 2 * dy - 2 * dx

        pixels.append((x, y))

    print("x =", x)
    print("y =", y)
    print("P =", p)
    print("pixel ->", (x, y))

    return pixels

### Main Program ###

x1 = int(input("Enter x1: "))
y1 = int(input("Enter y1: "))
x2 = int(input("Enter x2: "))
y2 = int(input("Enter y2: "))

# Call Bresenham function
pixels = bresenham_line(x1, y1, x2, y2)

### Plotting ###
x_pixels = [p[0] for p in pixels]
y_pixels = [p[1] for p in pixels]
plt.figure(figsize=(8, 6))

# Ideal Mathematical Line
plt.plot(
    [x1, x2],
    [y1, y2],
    label="Ideal Mathematical Line"
)

# Generated Pixels
plt.scatter(
    x_pixels,
    y_pixels,
    s=100,
    marker="s",
    color="red",
    label="Bresenham Generated Pixels"
)

# End Points
plt.scatter(
    [x1, x2],
    [y1, y2],
    s=150,
    marker="o",
    color="blue",
    label="End Points"
)

# Pixel coordinate labels
for x, y in pixels:
    plt.text(
        x + 0.08,
        y + 0.08,
        f"({x},{y})",
        fontsize=10
    )

plt.xticks(range(min(x1, x2) - 1, max(x1, x2) + 2))
plt.yticks(range(min(y1, y2) - 1, max(y1, y2) + 2))

plt.grid(True)
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.title("Bresenham's Straight Line Algorithm")
plt.legend()
plt.show()
```

Output:

Enter x1: 2
Enter y1: 3
Enter x2: 12
Enter y2: 15

--- Bresenham's Straight Line Algorithm ---

Starting Point: (2, 3)

Ending Point: (12, 15)

Delta X = 10

Delta Y = 12

Initial Decision Parameter P = 14

Starting Pixel -> (2, 3)

Case 2: $P \geq 0$

x = 3

y = 4

P = 18

pixel -> (3, 4)

Case 2: $P \geq 0$

x = 4

y = 5

P = 22

pixel -> (4, 5)

Case 2: $P \geq 0$

x = 5

y = 6

P = 26

pixel -> (5, 6)

Case 2: $P \geq 0$

x = 6

y = 7

P = 30

pixel -> (6, 7)

Case 2: $P \geq 0$

x = 7

y = 8

P = 34

pixel -> (7, 8)

Case 2: $P \geq 0$

x = 8

y = 9

P = 38

pixel -> (8, 9)

Case 2: $P \geq 0$

x = 9

y = 10

P = 42

pixel -> (9, 10)

Case 2: $P \geq 0$

x = 10

y = 11

P = 46

pixel -> (10, 11)

Case 2: $P \geq 0$

x = 11

y = 12

P = 50

pixel -> (11, 12)

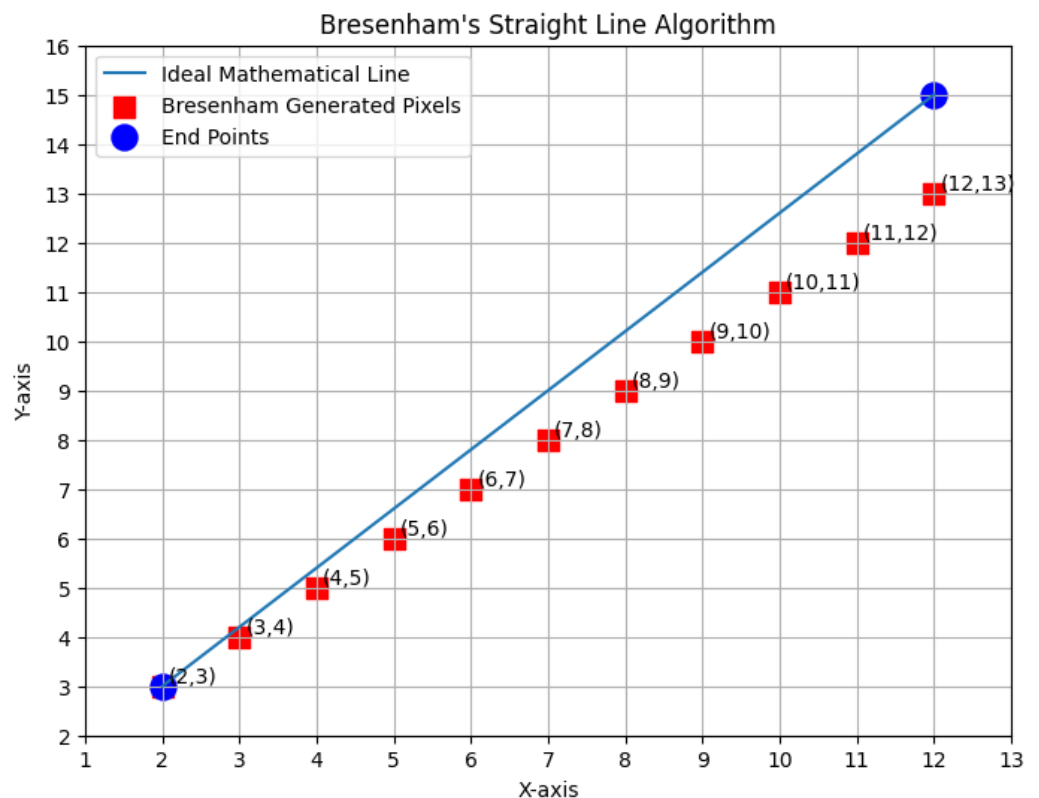
Case 2: $P \geq 0$

x = 12

y = 13

P = 54

pixel -> (12, 13)



Experiment 4: Implementation of Bresenham's Circle Drawing Algorithm.

Source Code:

```
import matplotlib.pyplot as plt

def bresenham_circle(R):
    pixels = []

    # Initial values
    x = 0
    y = R
    p = 3 - 2 * R

    print("\n--- Bresenham's Circle Algorithm ---")
    print("Radius:", R)
    print("Initial x =", x)
    print("Initial y =", y)
    print("Initial P =", p)

    # Generate points for one octant
    while x <= y:

        # 8 symmetric points
        pixels.append((x, y))
        pixels.append((-x, y))
        pixels.append((x, -y))
        pixels.append((-x, -y))
        pixels.append((y, x))
        pixels.append((-y, x))
        pixels.append((y, -x))
        pixels.append((-y, -x))

        print("\nCurrent Point:", (x, y))
        print("P =", p)
        # Case 1: P < 0
        if p < 0:
            print("Case 1: P < 0")

            # xi+1 = xi + 1
            x = x + 1

            # yi+1 = yi
            y = y

            # Pi+1 = Pi + 4xi + 6
            p = p + 4 * x + 6

        # Case 2: P >= 0
        else:
            print("Case 2: P >= 0")

            # xi+1 = xi + 1
            x = x + 1

            # yi+1 = yi - 1
            y = y - 1

            # Pi+1 = Pi + 4(xi - yi) + 10
            p = p + 4 * (x - y) + 10

        print("Next x =", x)
        print("Next y =", y)
        print("Next P =", p)

    return pixels

### Main Program ###

R = int(input("Enter Radius: "))

pixels = bresenham_circle(R)

### Plotting Circle ###

x_pixels = [p[0] for p in pixels]
y_pixels = [p[1] for p in pixels]

plt.figure(figsize=(8, 8))

plt.scatter(
    x_pixels,
    y_pixels,
    s=100,
    marker="s",
    color="red",
    label="Circle Pixels"
)

plt.axhline(0)
plt.axvline(0)

plt.grid(True)

plt.xlabel("X-axis")
plt.ylabel("Y-axis")

plt.title("Bresenham's Circle Drawing Algorithm")

plt.axis("equal")
plt.legend()

plt.show()
```

Output:

Enter Radius: 11

--- Bresenham's Circle Algorithm ---

Radius: 11

Initial $x = 0$

Initial $y = 11$

Initial $P = -19$

Current Point: (0, 11)

$P = -19$

Case 1: $P < 0$

Next $x = 1$

Next $y = 11$

Next $P = -9$

Current Point: (1, 11)

$P = -9$

Case 1: $P < 0$

Next $x = 2$

Next $y = 11$

Next $P = 5$

Current Point: (2, 11)

$P = 5$

Case 2: $P \geq 0$

Next $x = 3$

Next $y = 10$

Next $P = -13$

Current Point: (3, 10)

$P = -13$

Case 1: $P < 0$

Next $x = 4$

Next $y = 10$

Next $P = 9$

Current Point: (4, 10)

$P = 9$

Case 2: $P \geq 0$

Next $x = 5$

Next $y = 9$

Next $P = 3$

Current Point: (5, 9)

$P = 3$

Case 2: $P \geq 0$

Next $x = 6$

Next $y = 8$

Next $P = 5$

Current Point: (6, 8)

$P = 5$

Case 2: $P \geq 0$

Next $x = 7$

Next $y = 7$

Next $P = 15$

Current Point: (7, 7)

$P = 15$

Case 2: $P \geq 0$

Next $x = 8$

Next $y = 6$

Next $P = 33$

